Task 1. Provision an EC2 instance named `MusicApp-Server`

For this section, use a separate project folder so it does not interfere with the previous Terraform configuration.

`main.tf`

None

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

data "aws_ami" "linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
}

resource "aws_instance" "music_app" {
```

```
  ami           = data.aws_ami.linux.id
  instance_type = "t2.micro"

  tags = {
    Name = "MusicApp-Server"
  }
}

output "public_ip" {
  value = aws_instance.music_app.public_ip
}
```

Run:

```
None
terraform init
terraform validate
terraform plan
terraform apply
terraform output public_ip
```

This provisions an EC2 instance in the Mumbai region, subject to instance-type availability and AWS account eligibility.

Task 2. Create a Node.js application and Dockerfile

Project structure:

```
None
music-playlist/
├── app.js
├── package.json
└── Dockerfile
```

app.js

None

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Welcome to My Playlist");
});

server.listen(3000, "0.0.0.0", () => {
  console.log("Playlist app listening on port 3000");
});
```

package.json

None

```
{
  "name": "music-playlist",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node app.js"
  }
}
```

Dockerfile

None

```
FROM node:22-alpine

WORKDIR /app

COPY package.json ./
COPY app.js ./

EXPOSE 3000
```

```
USER node

CMD ["npm", "start"]
```

Build and test locally:

```
None
docker build -t music-playlist .
docker run --rm -p 3000:3000 music-playlist
```

Open `http://localhost:3000`. You should see `Welcome to My Playlist`.

Task 3. Create a GitHub Actions workflow

Create `.github/workflows/deploy.yml`.

This workflow initializes Terraform, validates the configuration, applies the infrastructure, and builds the Docker image on pushes to `main`.

```
None
name: Music Playlist CI/CD

on:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  build-and-provision:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
```

```
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY
}}
          aws-region: ap-south-1

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform -chdir=terraform init

      - name: Terraform Validate
        run: terraform -chdir=terraform validate

      - name: Terraform Plan
        run: terraform -chdir=terraform plan

      - name: Terraform Apply
        run: terraform -chdir=terraform apply -auto-approve

      - name: Build Docker image
        run: docker build -t music-playlist:latest .
```

Required setup

- Put the Terraform configuration in the `terraform/` directory, or update the workflow paths.
- Add `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` as GitHub Actions secrets. Prefer short-lived AWS credentials through GitHub OIDC for a real project.
- Add a remote Terraform backend before using this workflow for shared infrastructure.
- The Docker image is built on the GitHub runner; this alone does not deploy it to EC2.

Task 4. Deploy the Docker container automatically to EC2

To deploy the built image, publish it to GitHub Container Registry (GHCR), then connect to EC2 and pull and run it.

Add a security group to Terraform that allows TCP port 3000 only from the intended client CIDRs, and configure SSH access from a restricted IP. Also configure the EC2 instance with Docker installed and the required permissions.

Add the following steps after the Docker build step in the workflow above:

None

```
      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Push Docker image
        run: |

IMAGE="ghcr.io/${GITHUB_REPOSITORY,,}/music-playlist:latest"
          docker tag music-playlist:latest "$IMAGE"
          docker push "$IMAGE"

      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USER }}
          key: ${{ secrets.EC2_SSH_PRIVATE_KEY }}
          script: |
            docker pull ghcr.io/${{ github.repository
}}/music-playlist:latest
            docker stop music-playlist || true
            docker rm music-playlist || true
            docker run -d --restart unless-stopped \
              --name music-playlist \
              -p 3000:3000 \
```

```
              ghcr.io/${{ github.repository
}}/music-playlist:latest
```

Configure these GitHub secrets:

- `EC2_HOST`: the EC2 public DNS name or IP.
- `EC2_USER`: the correct operating-system username.
- `EC2_SSH_PRIVATE_KEY`: the private key used for deployment.

Important corrections for a working deployment:

1. Install Docker on the EC2 instance and allow the deployment user to execute Docker commands.
2. For a private GHCR image, authenticate the EC2 instance to GHCR before running `docker pull`, using a least-privilege token stored securely on the instance.
3. Ensure the EC2 security group permits SSH only from the GitHub runner's approved access path. Prefer a controlled self-hosted runner, VPN, or AWS Systems Manager over opening SSH broadly.
4. The container must listen on port 3000. Open `http://EC2_PUBLIC_IP:3000` only if the corresponding network rule allows your client.

For production, use an immutable image tag such as the Git commit SHA instead of `latest`, and consider a deployment platform that avoids exposing the application directly on a public high port.

Task 5. Add an S3 bucket

Add the following to `main.tf`:

```
None
resource "aws_s3_bucket" "playlist" {
  bucket = "my-playlist-app-bucket-replace-with-unique-suffix"

  tags = {
    Name  = "my-playlist-app-bucket"
    Owner = "Your Name"
  }
}
```

S3 bucket names must be globally unique. Replace the example name with a unique name before applying.

For secure storage, keep public access blocked and enable encryption and versioning. Then run:

```
None
terraform fmt
terraform validate
terraform plan
terraform apply
```